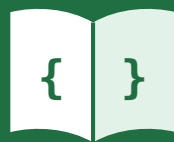

Module 05 — Spring JDBC and PostgreSQL

Exercises

Spring Boot Edu



2026-09-23

Contents

How to Work	2
Exercise 1 — A Migration for Reviews (Easy)	2
Exercise 2 — A Review Repository and Batch Inserts (Medium)	3
Exercise 3 — All or Nothing: Importing Reviews (Hard)	3
Bonus Challenge (Optional)	4

How to Work

1. The starter code is in `modules/05-data-jdbc-postgres/exercise/`. Find the TODO comments and the file `db/migration/TODO-exercise-1.txt`.
2. The tests run against a real PostgreSQL (Testcontainers). **Docker must be running.**
3. Run the tests:

```
./mvnw -Pexercises -pl modules/05-data-jdbc-postgres/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/05-data-jdbc-postgres/solution/`.

! Important

Do the exercises in order: Exercises 2 and 3 use the review table that you create in Exercise 1.

Exercise 1 — A Migration for Reviews (Easy)

Goal: add a table through a Flyway migration, with its rules protected by the database.

Tasks: create the file `exercise/src/main/resources/db/migration/V3__create_review.sql`. The review table:

Column	Rule
<code>id</code>	<code>bigserial</code> , primary key
<code>book_id</code>	Not null, references <code>book(id)</code> . Reviews are deleted together with their book
<code>stars</code>	<code>smallint</code> , not null, only 1-5
<code>comment</code>	<code>text</code>
<code>created_at</code>	<code>timestamp</code> , not null, defaults to the current time

Hints:

- `V1__create_schema.sql` from lesson section 3.2 is a good model.
- references `book (id)` on delete cascade, check (stars between 1 and 5), default `now()`.
- The file name contains **two** underscores: `V3__create_review.sql`.

Acceptance criteria: all 6 tests in `Exercise1Test` pass.

Estimated time: 20 minutes

Exercise 2 — A Review Repository and Batch Inserts (Medium)

Goal: write CRUD with `JdbcClient` and insert many rows in a single batch.

Tasks (`exercise2.ReviewRepository`):

- TODO 2a — `add(review)`: insert the review and return the generated id. The book is given by ISBN.
- TODO 2b — `findByIsbn(isbn)`: the book's reviews, newest first, mapped to `Review` records.
- TODO 2c — `averageStars(isbn)`: the average rating. An empty `Optional` if there are no reviews.
- TODO 2d — `addAll(reviews)`: insert all reviews in **one batch** and return the update counts.

Hints:

- The book id can be looked up inside the INSERT: (`select id from book where isbn = :isbn`).
- `.paramSource(record)` takes the parameters from the record's components. Use a `KeyHolder` for the generated key (lesson section 3.3).
- In PostgreSQL, `avg()` returns numeric and is NULL when there are no rows. Cast with `::float8` and handle the NULL with an `Optional`.
- `JdbcClient` has no batch API. Use `NamedParameterJdbcTemplate.batchUpdate(sql, SqlParameterSourceUtils.createBatch(list))`.

Acceptance criteria: all 3 tests in `Exercise2Test` pass.

Estimated time: 40 minutes

Exercise 3 — All or Nothing: Importing Reviews (Hard)

Goal: make an import a single transaction, and record its outcome even when the transaction is rolled back.

Tasks (package `exercise3`):

- TODO 3a — `ReviewImportService.importAll`: either all reviews are saved, or none.
- TODO 3b — on success write SUCCESS and "<n> reviews" to the `ImportLog`. On failure write FAILED and the exception's simple class name, then rethrow the failure.
- TODO 3c — `ImportLog.record`: this row must be permanent even when the caller's transaction rolls back.

Hints:

- Lesson section 3.5: `@Transactional` and `@Transactional(propagation = Propagation.REQUIRES_NEW)`.
- An invalid rating (e.g. 9) hits the database constraint and arrives as a `RuntimeException`.
- The tests use `@SpringBootTest` and really commit. They clean up their own data.

Acceptance criteria: both tests in `Exercise3Test` pass.

Estimated time: 40 minutes

Bonus Challenge (Optional)

Model reviews with Spring Data JDBC: the `Book` aggregate holds its reviews as a `@MappedCollection`. Write a `@DataJdbcTest` that saves and loads a book together with its reviews. Compare it with the `JdbcClient` version: which fits which situation better?